



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Master Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital
Universidad Politécnica de Valencia

Biometría: Filtros, Curvas ROC y Detector facial Viola Jones

TRABAJO TECNOLOGÍAS DEL LENGUAJE HUMANO

Máster Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital

Gómez Corral, Miquel

CAPÍTULO 1

Introducción

$$(s_3, q_1) \xrightarrow{\textit{Caminar}} (s_4, q_1) \qquad (s_3, q_1) \xrightarrow{\textit{Entregar}} (s_4, q_2)$$

En este trabajo se ha realizado la implementación de tres de los ejercicios propuestos para la asignatura de Biometría: Filtros faciales, cálculos relacionados con la curva ROC y el detector facial Viola Jones.

De filtros faciales, se han implementado la normalización global y local por media y desviación típica únicamente. No se ha pretendido implementarlas todas pero sí las más relevantes.

Con respecto a la curva ROC, se ha implementado en un *notebook* todo el cálculo de falsos positivos (FP) y falsos negativos (FN) dado un umbral, así como el FN para un FP y viceversa. Hay visualizaciones de la curva ROC y el cálculo AUC.

Para la recreación del detector facial Viola Jones, se han implementado todas las partes mencionadas en el *paper* original: extracción de características Haar, selección de características con AdaBoost, clasificación en cascada, *hard negative mining* y varias optimizaciones adicionales. Una demo en tiempo real del detector puede verse en <https://youtu.be/JnWS4aAlpVo>.

Hardware utilizado

Los experimentos se han realizado en mi máquina personal: GPU NVIDIA RTX 5070 (12 GB VRAM), procesador AMD Ryzen 9 9000X, 64 GB de RAM y Ubuntu 24.04 LTS.

Sin embargo, dado el alto coste espacial de Viola Jones, se ha optado por realizar el experimento grande en el clúster Tensor, usando 250 GB de RAM y 32 CPUs. Los experimentos pequeños con unas 15k muestras se han podido realizar en la máquina personal sin problemas; los demás, en el clúster.

1.1 Código entregado

- `notebooks/Ej1-Filtres.ipynb`: Implementa todo lo relacionado con los filtros faciales.
- `notebooks/Ej2-CurvaRoc.ipynb`: Implementa todo lo relacionado con la curva ROC.
- `notebooks/Ej3-TrainViolaJones.ipynb`: Muestra todos los pasos necesarios para cargar los datos, extraer las características Haar, seleccionar las mejores características con AdaBoost y entrenar el clasificador en cascada para un reconocimiento facial basado en el *paper* de Viola y Jones.
- Repositorio y otros notebooks: El repositorio cuenta con varios scripts para el desarrollo. A través de `/app/main.py` se pueden lanzar las dos funciones principales: activar la cámara para usar el detector y otra para entrenar una instancia del modelo. Cuenta con más notebooks para la visualización de diferentes aspectos del proyecto en `/notebooks`. Más detalles sobre cómo usar el repositorio en el README.md incluido en el mismo.

CAPÍTULO 2

Filtros

Mencionar en esta sección, que en el contexto de la detección facial, el preprocesamiento de las imágenes es fundamental para garantizar la robustez del sistema frente a variaciones de iluminación y contraste. En este trabajo se ha implementado un filtro de normalización por media y desviación estándar local.

La normalización local se define, para cada píxel (i, j) , como:

$$I_{\text{norm}}(i, j) = \frac{I(i, j) - \mu_W(i, j)}{\sigma_W(i, j) + \epsilon}, \quad (2.1)$$

donde $\mu_W(i, j)$ y $\sigma_W(i, j)$ son la media y la desviación estándar calculadas en una ventana W centrada en el píxel, y ϵ es una constante de regularización para evitar divisiones por cero.

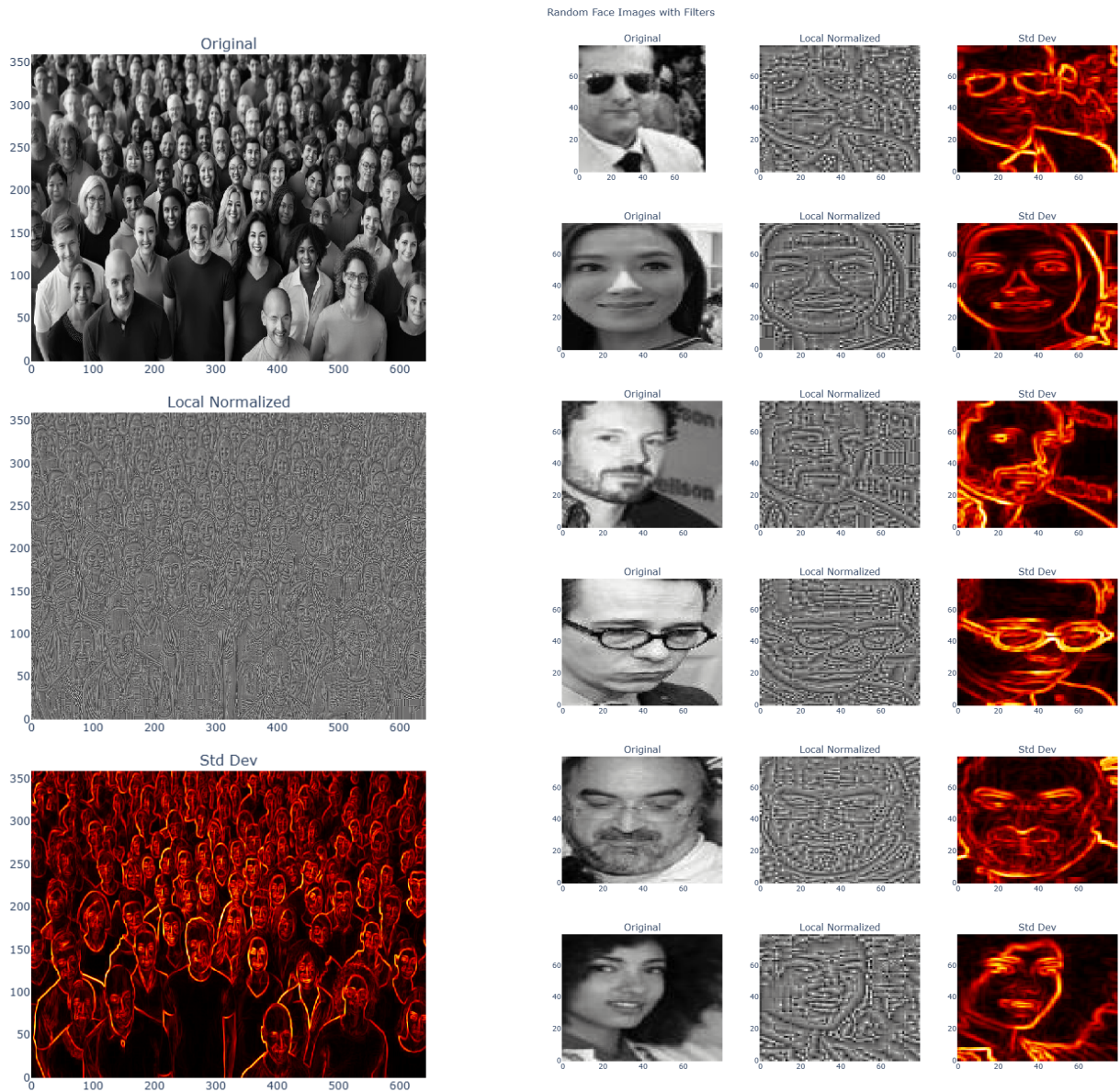
Para hacer este cálculo de las medias y la desviación estándar computacionalmente eficiente, se utilizan *imágenes integrales*. A partir de la imagen original I , se construye la imagen integral S y la imagen integral de cuadrados S_2 , de modo que la suma (y la suma de cuadrados) de cualquier región rectangular puede obtenerse en tiempo constante con solo cuatro accesos a memoria. Esto permite calcular la desviación estándar local para todos los píxeles de forma vectorizada, sin necesidad de recorrer explícitamente cada ventana.

En la Figura 2.1 se muestran los resultados de aplicar este filtro sobre dos tipos de imágenes: una fotografía de multitud (vista general) y varios recortes de caras individuales. En ambos casos se presentan tres vistas: la **imagen original** en escala de grises, la **versión localmente normalizada** y el **mapa de desviación estándar local** (donde los valores altos, en tonos cálidos, indican mayor variabilidad textural, típicamente asociada a bordes y contornos).

Como se observa en la Figura 2.1a, la normalización local resalta los contornos faciales incluso en condiciones de iluminación desuniforme, mientras que el mapa de desviación estándar evidencia las regiones de mayor contraste, como los bordes de las caras y los rasgos faciales.

En el caso de las caras individuales de la Figura 2.1b, el efecto es más pronunciado: la imagen normalizada reduce las diferencias en bruto entre muestras, y el mapa de desviación estándar destaca de manera consistente los ojos, la nariz, la boca y el contorno de la cara, lo cual resulta especialmente útil para la extracción posterior de características Haar.

Nota: la imagen normalizada puede parecer ruidosa a baja resolución; se recomienda ampliar el PDF para apreciar los detalles.



(a) Imagen de multitud con sus correspondientes filtros. De arriba abajo: original, normalizada localmente y desviación estándar.

(b) Malla de caras individuales mostrando el efecto de los filtros. Cada fila presenta, de izquierda a derecha: original, normalizada localmente y desviación estándar.

Figura 2.1: Resultados de la normalización local y el cálculo de desviación estándar sobre imágenes de multitud (izquierda) y caras individuales (derecha).

CAPÍTULO 3

Curva ROC

La curva ROC es una herramienta para evaluar un sistema de clasificación, permite visualizar el compromiso entre TPR, *True Positive Rate* y la FPR, *False Positive Rate* al variar el umbral de decisión del clasificador.

En este trabajo se ha implementado un *notebook* que, dado un conjunto de puntuaciones de clientes e impostores, calcula estos ratios de forma exhaustiva para todos los umbrales posibles.

Se han utilizado dos conjuntos de datos, denominados A y B, cada uno con 2 990 muestras (clientes e impostores mezclados). Las puntuaciones representan la confianza del sistema de que una muestra pertenece a un cliente legítimo. Para un umbral dado θ , se definen:

- **Falsos Positivos (FP):** impostores clasificados erróneamente como clientes ($\text{score} > \theta$).
- **Falsos Negativos (FN):** clientes rechazados erróneamente ($\text{score} \leq \theta$).

A partir de estos valores se computan las tasas FPR y FNR para cada umbral, permitiendo trazar la curva ROC ($\text{TPR} = 1 - \text{FNR}$ frente a FPR). Además, se calcula el área bajo la curva (AUC) mediante integración numérica por el método del trapecio (que se calcula en tiempo lineal, en lugar de cuadrático comparando todos los clientes con los impostores como hacemos en clase con pocos datos), obteniendo una métrica global de la capacidad discriminativa del sistema.

Métrica	Dataset A	Dataset B	Umbrales (θ_A / θ_B)
AUC	0,8831	0,8830	—
D prime	0.7597	0.8732	—
FN = FP	20,14 %	20,07 %	0,0503 0,0444
FP para FN = 1 %	80,13 %	68,00 %	0,0071 0,0000
FP para FN = 5 %	57,56 %	53,65 %	0,0202 0,0048
FP para FN = 20 %	20,45 %	20,13 %	0,0500 0,0443
FN para FP = 1 %	62,73 %	64,55 %	0,1474 0,2999
FN para FP = 5 %	39,58 %	44,55 %	0,0826 0,1444
FN para FP = 20 %	20,14 %	20,42 %	0,0504 0,0450

Tabla 3.1: Resultados principales del análisis ROC para los datasets A y B.

Los resultados, resumidos en la Tabla 3.1, muestran que ambos datasets presentan un comportamiento muy similar, con un AUC cercano a 0,88, lo que indica una buena capacidad de discriminación del sistema. No obstante, el punto de igual error (donde $\text{FN} \approx \text{FP}$) se sitúa en torno al 20 %, lo cual

sugiere que, para aplicaciones biométricas exigentes, sería necesario ajustar el umbral en función de si se prioriza minimizar los falsos positivos o los falsos negativos.

Por ejemplo, si se fija un umbral de $\theta = 0,10$, el dataset A presenta una tasa de FP del 2,63 % y una tasa de FN del 48,04 %, mientras que el dataset B arroja valores de 8,53 % y 34,62 %, respectivamente. Estas diferencias evidencian que la distribución de puntuaciones varía entre conjuntos de datos, por lo que la elección del umbral debe adaptarse al escenario de aplicación concreto.

Para obtener una tasa de FN en torno al 1 % (apenas rechazar clientes) es necesario reducir drásticamente el umbral. En el dataset A se requiere un umbral de 0,0071, mientras que en el dataset B ni siquiera existe un umbral positivo que consiga ese FN, por lo que el valor más cercano es $\theta = 0,0000$, equivalente a aceptar toda puntuación mayor que cero. Ni siquiera con $\theta = 0$ se alcanza un FP del 100 %, ya que aproximadamente un 31 % de los impostores tiene puntuación exactamente cero y, por tanto, son correctamente rechazados.

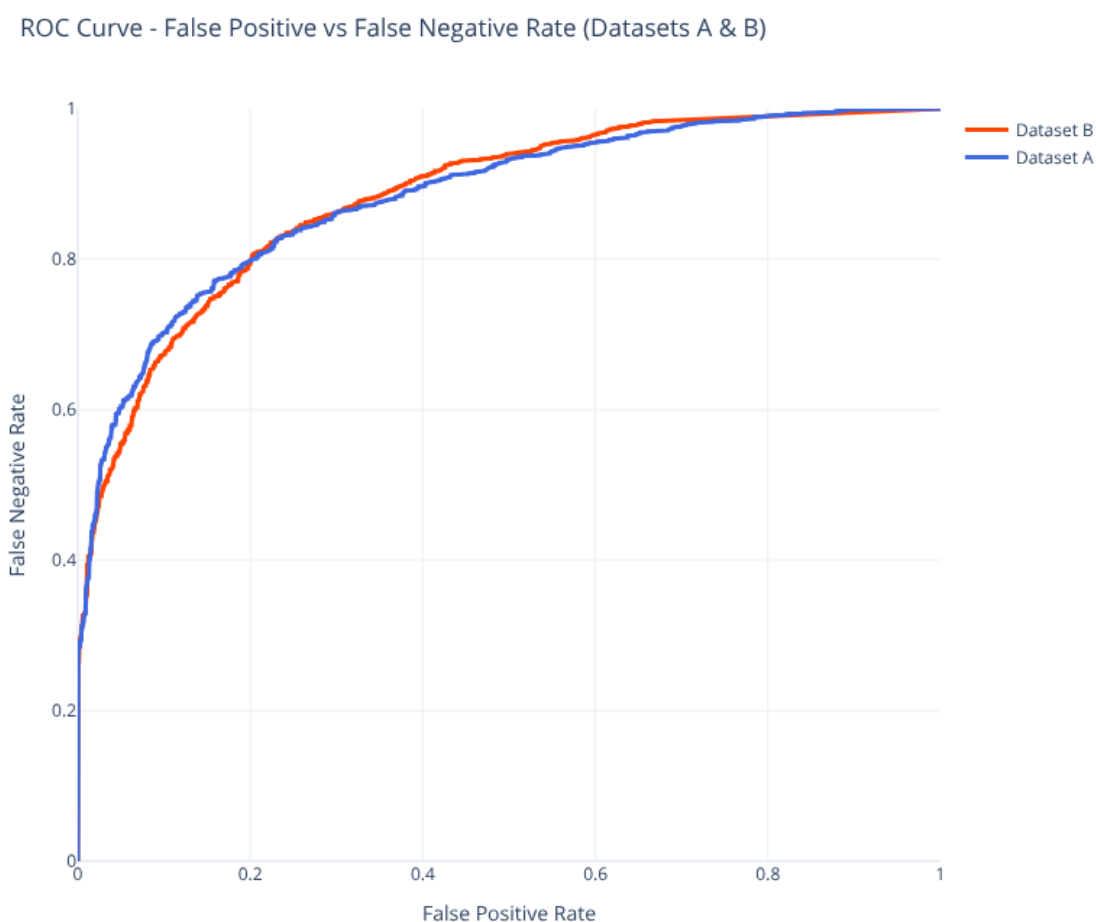


Figura 3.1: Curva ROC para los datasets A y B.

Ambos datasets muestran curvas prácticamente solapadas, lo que confirma que el sistema de puntuación se comporta de forma similar en ambos conjuntos. El AUC cercano a 0,88 indica una capacidad de discriminación aceptable, aunque la zona de bajo FP revela una caída pronunciada de la TPR, reflejando la dificultad de mantener una alta tasa de acierto cuando se exige una tasa de falsos positivos muy reducida.

CAPÍTULO 4

Viola Jones

La implementación sigue fielmente el *paper* original de Viola y Jones, con varias optimizaciones adicionales. Las fases principales son: (1) obtención de datos, (2) listado de características Haar, (3) selección de características con AdaBoost, (4) *hard negative mining* y (5) clasificación en cascada.

4.1 Obtención de datos

4.1.1. Dataset de NO caras

Las muestras negativas se extraen del conjunto open-images-v7 mediante *fiftyone*. Para garantizar la ausencia de caras, se filtran imágenes usando etiquetas que casi siempre implican ausencia de personas (*Ant, Apple, Banana...*) y se excluyen etiquetas que puedan incluirlas (*Human face, Human arm...*). Las imágenes restantes se filtran con el detector OpenCV para eliminar cualquier cara residual. El resultado son más de 128 000 imágenes sin caras.

El conjunto se divide en 90 % entrenamiento y 10 % test. Del 10 % de test se generan 2,5 M de *crops* estáticos para monitorizar la tasa de FP a lo largo del entrenamiento.



Figura 4.1: Ejemplos de imágenes sin caras extraídas de Open Images V7.

4.1.2. Dataset de caras

Las caras del *dataset* original no están centradas ni alineadas (Figura 4.2, imagen izquierda). Se procesan con el detector de OpenCV, conservando solo los *crops* con confianza superior a 8 (umbral normal: 5), lo que garantiza centrado en ojos y nariz. De 75 k imágenes se obtienen 26 k caras de 24×24 px; con su versión espejo, más de 52 k. Se reserva el 10 % para test.

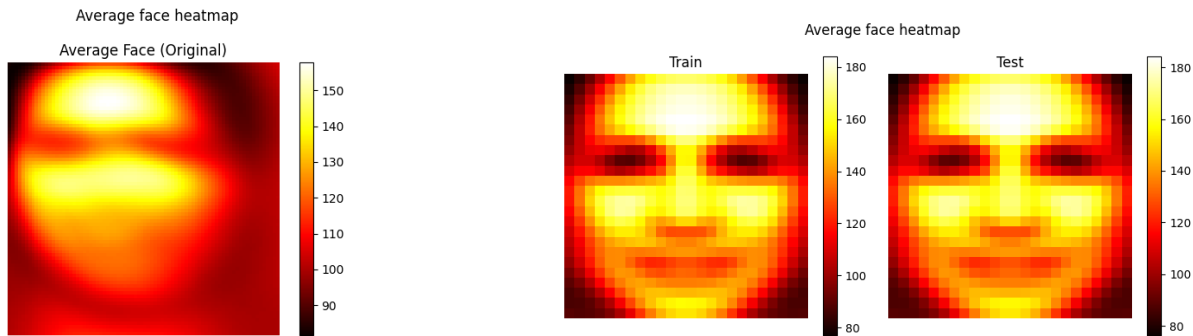


Figura 4.2: Cara media antes (izquierda, borrosa) y después (derecha) de procesar el dataset: los *crops* centrados producen una cara nítida.

4.2 Características Haar y AdaBoost

4.2.1. Listado de características

Se recorren iterativamente todas las posiciones y tamaños de los filtros Haar sobre *crops* de 24×24 px. Los filtros invertidos se omiten porque AdaBoost determina el signo del peso asociado. Todos los valores se calculan mediante **imagen integral**.

Para reducir el espacio de características de 170 k a 81 k, se limita el tamaño mínimo de cada filtro: horizontales 6×4 , verticales 4×6 , 3 horizontales 9×4 , 3 verticales 4×9 , diagonales 6×6 . Se añade además un filtro tipo marco (cuadrado con cuadrado concéntrico interior, mínimo 6×6), sumando 2 k características más.

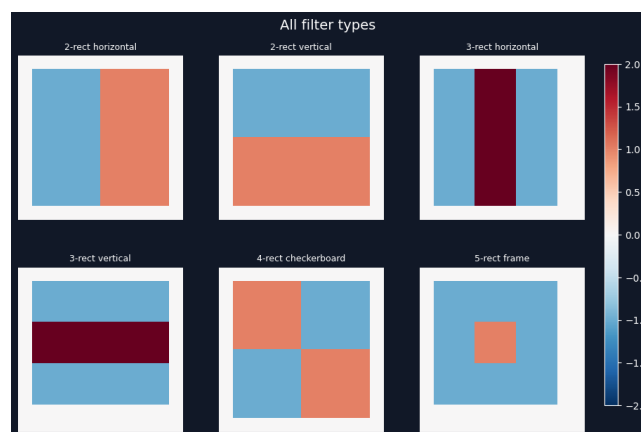


Figura 4.3: Tipos de filtros Haar utilizados, incluido el filtro cuadrado concéntrico propuesto.

4.2.2. Selección con AdaBoost

Se emplea una implementación personalizada de AdaBoost (con asistencia de Claude) con interfaz compatible con *scikit-learn*, especializada en *stumps* y con búsqueda paralela del mejor *split*

por característica. Esta versión es significativamente más rápida que la estándar dado el volumen de características y muestras.

Tras cada *stump*, se ajusta el umbral de etapa para garantizar $\text{TPR} \geq 0,999$ (o $\geq 0,99$ según configuración). Si el FP resultante alcanza el objetivo del 50 %, la etapa concluye; de lo contrario, se continúa hasta un máximo de 500 *stumps*. La elección de la TPR mínima tiene un efecto acumulativo importante: tras 30 etapas, una TPR de 0,99 por etapa resulta en $0,99^{30} = 0,74$ de caras conservadas globalmente, frente a $0,999^{30} = 0,97$ con la configuración más estricta. Esto crea un compromiso directo entre la exigencia por etapa y la tasa de detección final.

Se experimenta también con una **tasa de FP progresiva**: etapas 1–2 con objetivo 15 %, etapas 3–5 con 25 %, etapas 6–10 con 40 %, y etapas 11+ con 50 %. El objetivo es que las primeras etapas, donde el *hard mining* es menos exigente, eliminen más negativos aunque requieran más *stumps*.

4.3 Hard Mining y Clasificador en Cascada

4.3.1. Hard negative mining

Para cada etapa, las muestras negativas se obtienen pasando imágenes de open-images-v7 por la cascada acumulada: los *crops* que superan todas las etapas previas son los falsos positivos más difíciles y conforman el conjunto negativo de entrenamiento. Este paso es esencial: no sirve precomputar un gran pool estático de negativos, ya que en pocas etapas el clasificador aprende a descartarlos y se queda sin muestras negativas útiles. Con *hard mining* dinámico, cada etapa siempre se enfrenta a los casos más difíciles del momento.

Preservación de FP heredados: Los FP heredados de etapas anteriores se preservan y concatenan con los nuevos para evitar el olvido de patrones ya aprendidos.

Diversidad: Para asegurar diversidad, se limita la extracción a un máximo del 10 % del total de *crops* por imagen. Se pueden extraer menos si la imagen no tiene suficientes *crops* que superen las etapas previas, pero no más para evitar sobre-representar ciertas imágenes.

4.3.2. Clasificador en cascada

El script `generate_all_stages` encadena las etapas, monitorizando la tasa global de FP sobre los 2,5 M de *crops* de test tras cada nueva etapa. El entrenamiento se detiene cuando se descarta la totalidad de los *crops* de test o se alcanza el número máximo de etapas. El estado completo se guarda en *checkpoint* tras cada etapa para permitir reanudar el entrenamiento si se interrumpe.

4.4 Inferencia

La versión de producción del clasificador (`app/src/model/cascade_classifier.py`) está vectorizada y paralelizada con `numba` (`@njit(parallel=True)`), logrando evaluación en tiempo real. Para detectar caras a múltiples escalas se implementa una **pirámide de escalas** (factor de submuestreo 0,8 por defecto). Opcionalmente, se puede reducir la imagen de entrada a la mitad antes de iniciar la pirámide (`halve_size_factor`), lo que acelera significativamente la inferencia a costa de no detectar caras por debajo de un tamaño mínimo de *crop*.

NMS: Las detecciones solapadas se consolidan con `cv2.groupRectangles` (NMS, $\epsilon = 0,2$).

Serialización: El modelo se serializa con `xml` compatible con OpenCV para facilitar la comparación y uso de etapas preentrenadas.

CAPÍTULO 5

Conclusiones y Resultados (Detector)

Se realizan cinco experimentos variando dos parámetros: TPR mínima por etapa (0,999 / 0,99) y uso o no de tasa de FP progresiva. Los cuatro primeros usan 15 000 muestras positivas y 15 000 negativas por etapa; el quinto (pfp_999_all_data) usa todas las muestras disponibles, entrenado en el clúster Tensor. Se muestra su progreso a lo largo de las etapas y resultados finales en test (FP = 0 % en todos los casos).

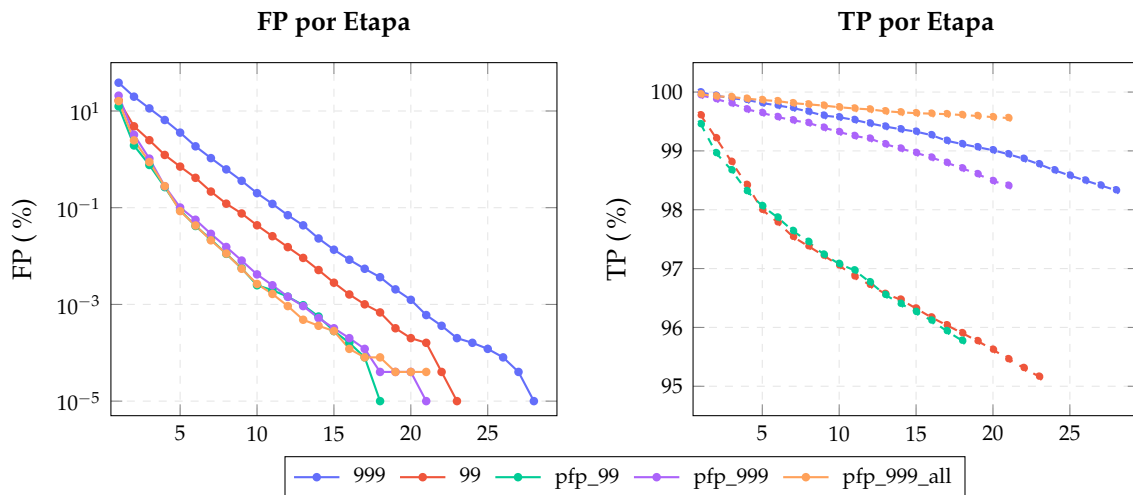


Figura 5.1: Progresión de la tasa de FP (escala logarítmica, línea sólida) y TPR (línea discontinua) a lo largo de las etapas del clasificador en cascada para cada configuración.

Modelo	Descripción	Etapas	TPR final
balanced_999	$\text{TPR} \geq 0,999$, FP fijo	28	0,9833
balanced_99	$\text{TPR} \geq 0,99$, FP fijo	23	0,9516
pfp_99	$\text{TPR} \geq 0,99$, FP progresivo	18	0,9577
pfp_999	$\text{TPR} \geq 0,999$, FP progresivo	21	0,9841
pfp_999_all*	$\text{TPR} \geq 0,999$, FP progresivo	21	0,9956

Tabla 5.1: Configuraciones y resultados. Todos los modelos alcanzan FP = 0 % en test. (*Con todos los datos).

Modelo	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21	S22	S23	S24	S25	S26	S27	Total
99	2	5	5	8	6	12	15	23	26	36	37	40	54	62	67	77	90	93	123	125	141	157	178	-	-	-	-	-	1382
999	4	6	8	11	15	24	30	38	44	56	70	76	93	112	128	141	157	187	202	222	256	284	330	346	367	419	475	500	4601
pfp_99	4	10	13	22	39	36	47	56	68	87	84	91	98	112	133	137	159	178	-	-	-	-	-	-	-	-	-	-	1374
pfp_999	7	23	38	59	86	85	109	133	155	181	186	218	225	246	302	317	355	395	414	452	500	-	-	-	-	-	-	-	4486
pfp_999_all	11	34	52	85	136	153	188	250	302	371	370	443	500	500	500	500	500	500	500	500	500	-	-	-	-	-	-	-	6895

Tabla 5.2: Número de filtros Haar seleccionados por etapa en cada configuración.

Los modelos con $\text{TPR} \geq 0,999$ requieren notablemente más filtros (balanced_999: 4601 frente a 1382 de balanced_99). En pfp_999_all, 9 de las 21 etapas alcanzan el máximo de 500 *stumps* sin llegar al objetivo de FP, lo que indica que la mayor diversidad de datos requeriría un límite más alto de *stumps* para una convergencia más rápida.

5.0.1. Filtros de la primera etapa

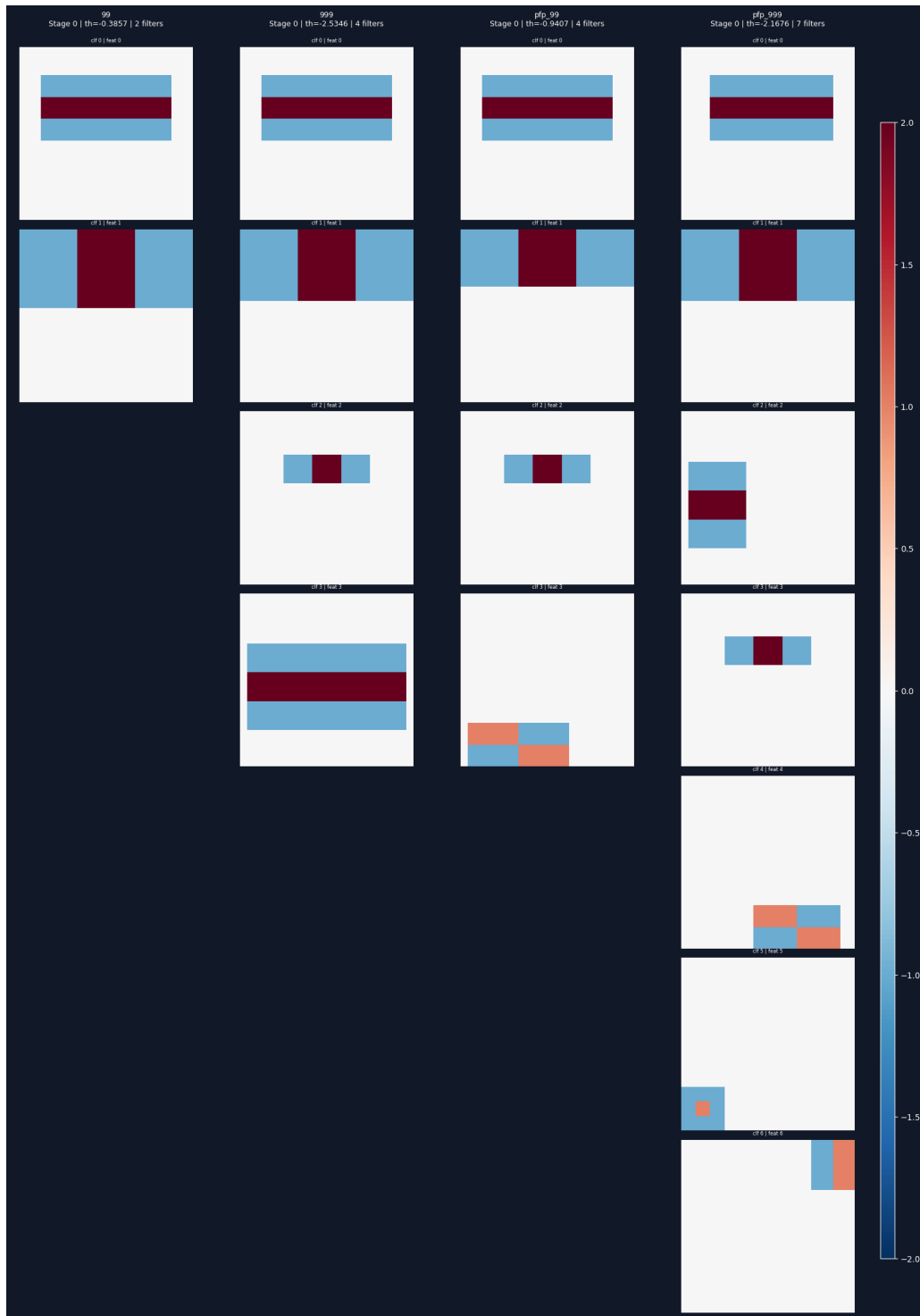


Figura 5.2: Filtros Haar de la primera etapa para cada configuración (de izquierda a derecha: nopfp_99, nopfp_999, pfp_99, pfp_999, pfp_999_all_data).

Los primeros 2–3 filtros son prácticamente idénticos en todas las configuraciones (contraste ojos-frente, nariz-mejillas), confirmando que estas son las características faciales más discriminativas. El filtro cuadrado concéntrico propuesto apenas aparece en ninguna configuración, lo que indica que no aporta valor adicional sobre los filtros Haar clásicos.

5.0.2. Inferencia en tiempo real

Modelo	FPS	Observaciones
balanced_99	~72,3	Menor precisión
balanced_999	~60,5	Detección robusta
pfp_99	~69,0	Menor precisión
pfp_999	~59,3	Detección robusta
pfp_999_all	~53,1	Detección robusta

Tabla 5.3: Velocidad de inferencia en tiempo real para cada configuración.

Todos los modelos superan ampliamente los 30 FPS. Los modelos $TPR \geq 0,99$ alcanzan ~70 FPS frente a ~60 FPS de los de 0,999, con pfp_999_all en ~53 FPS. La tasa progresiva de FP no aporta ventaja en velocidad: aunque reduce el número de etapas, concentra más *stumps* en cada una, resultando en un número total prácticamente idéntico. La diferencia real está en la calidad de detección: los modelos 0,99 presentan fallos frecuentes con gafas y rotaciones de cabeza que los de 0,999 manejan con mayor robustez.

5.1 Conclusiones

- **$TPR \geq 0,999$ vs. 0,99:** La TPR mínima por etapa es el factor determinante del rendimiento final. Los modelos 0,999 son superiores en robustez (seguimiento en rotación, invarianza a gafas) con una penalización moderada en velocidad (~10 FPS) que no compromete la operación en tiempo real. pfp_999_all alcanza la TPR final más alta (0,9956) a 53 FPS.
- **Progressive FPR:** Un resultado relevante es que la tasa progresiva de FP no aporta ventaja en velocidad. Aunque reduce el número de etapas, concentra más *stumps* en cada una, y dado que las etapas anteriores procesan un mayor número de *crops* (todavía no descartados por la cascada), el coste computacional total resulta prácticamente idéntico o ligeramente superior: pfp_99 a 69,0 FPS frente a balanced_99 a 72,3; pfp_999 a 59,3 frente a balanced_999 a 60,5. La mejora en TPR es igualmente marginal ($<0,01$).
- **Escalado con datos:** pfp_999_all es el modelo más lento y con más filtros (6 895), con 9 etapas saturando el límite de 500 *stumps*. Su TPR desciende más gradualmente, lo que indica que mayor diversidad de datos dificulta la convergencia pero produce un clasificador más robusto. Aumentar el límite de *stumps* o el número de etapas podría mejorar aún más el resultado.
- **Filtros Haar:** Las características más discriminativas (contraste ojos-frente, nariz-mejillas) son consistentes en todas las configuraciones. El filtro cuadrado concéntrico propuesto no ha aportado valor adicional sobre los filtros clásicos.
- **Optimizaciones:** La paralelización del AdaBoost, la vectorización con numba y el *hard negative mining* con preservación de FP han sido esenciales para hacer viable el entrenamiento (45 s/*stump*) y alcanzar detección en tiempo real en todas las configuraciones.